

Flux 设计文档

双分支扩散Transformer：原生音视频联合生成

1. 项目概述

Flux 是由 TopoSeek Inc. 开发的双分支扩散 Transformer，用于原生音视频联合生成。它是受字节跳动 Seedance 架构启发的开源参考实现，支持 T2VA 和 I2VA 生成。

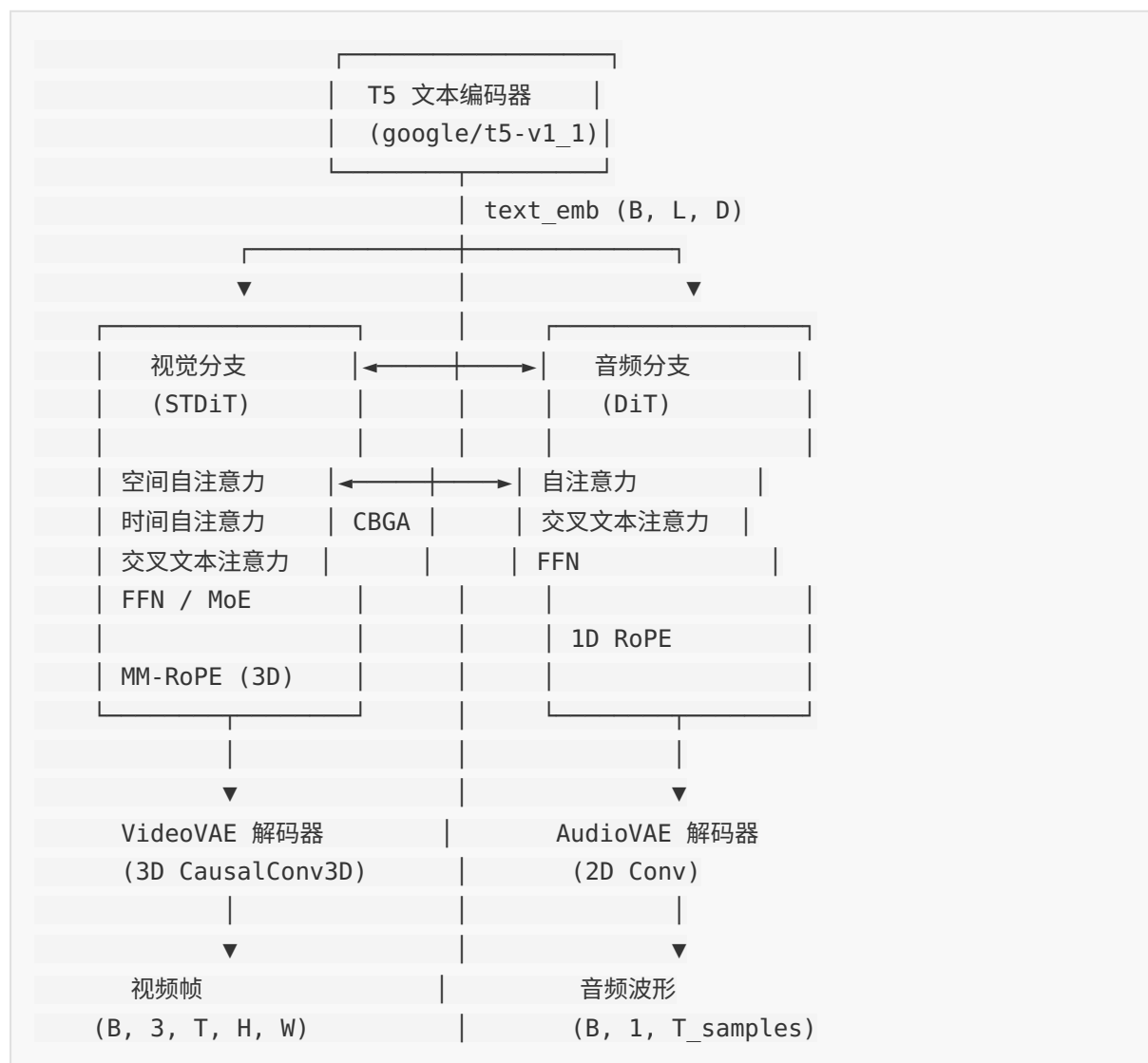
1.1 核心能力

能力	描述
T2VA	文本描述 → 联合视频 + 音频生成
I2VA	输入图像 + 文本 → 视频 + 音频（首帧条件注入）
多分辨率	256×256 训练，级联超分至 4K
长时生成	30s+ 长视频生成（级联时间扩展）
多尺度模型	Small (0.4B) / Base (1.6B) / 30B Dense / 30B MoE / 200B MoE
RLHF	5 维奖励模型 + PPO 强化学习微调
SFT	角色一致性 + 面部关键点 + 镜头控制监督微调

1.2 技术栈

- 语言: Python 3.10+
- 深度学习: PyTorch 2.10.0, CUDA 12.8
- 分布式: FSDP (FULL_SHARD), DeepSpeed ZeRO-3, torchrun
- 注意力后端: xformers > flash-attn > PyTorch SDPA（自动选择）
- 混合精度: bf16（主要），fp16（GradScaler 回退）
- 配置: OmegaConf YAML
- 监控: TensorBoard + Weights & Biases
- 包管理: uv（锁定依赖）

2. 系统架构



2.1 数据流概述



3. 模型组件

3.1 VideoVAE — 3D 视频自编码器

文件: [flux/models/video_vae/](https://github.com/fluxml/flux/blob/main/flux/models/video_vae/)

属性	值
输入	<code>(B, 3, T, H, W)</code> — RGB 视频帧
输出潜变量	<code>(B, 16, T/4, H/8, W/8)</code>
空间压缩	$8\times$ (三阶段 $2\times 2\times 2$)
时间压缩	$4\times$ (两阶段 $2\times 2\times 1$)
总压缩比	$8\times 8\times 4 = 256\times$
骨干网络	CausalConv3D ResNet
正则化	KL 散度 (对角高斯后验)
判别器	3D PatchGAN

设计要点: - **CausalConv3D:** 时间卷积为因果卷积——第 t 帧仅依赖 $\leq t$ 的帧，防止未来信息泄露 - **SDXL 初始化:** 支持从 SDXL VAE 初始化 2D 权重，2D 卷积核扩展为 3D (居中时间位置)，时间层零初始化 - **GroupNorm:** 32 组，与 SDXL 共享类似的空间归一化

3.2 AudioVAE — 2D 音频自编码器

文件: [flux/models/audio_vae/](https://github.com/fluxml/flux/blob/main/flux/models/audio_vae/)

属性	值
输入	梅尔频谱 <code>(B, 1, 80, T_frames)</code>
输出潜变量	<code>(B, 8, F_lat, T_lat)</code>
采样率	16 kHz
梅尔频带	80
跳步长度	256
骨干网络	2D Conv ResNet
正则化	KL 散度

3.3 DB-DiT — 双分支扩散 Transformer

文件: [flux/models/db_dit/db_dit.py](#)

核心模型，由以下组件构成：

3.3.1 整体结构

输入：

```
v_latent (B, 16, T, H, W)    → 视频 Patch Embed → v_tokens (B, N_v, D)
a_latent (B, 8, F, T_a)      → 音频 Patch Embed → a_tokens (B, N_a, D)
t (B,)                       → 时间步 Embed      → t_emb (B, D)
text_emb (B, L_text, D_text) → (保持原样)
```

输出：

```
v_pred (B, 16, T, H, W)  - 视频速度场
a_pred (B, 8, F, T_a)    - 音频速度场
```

3.3.2 DualBranchBlock — 双分支 Transformer 层

每个层按顺序执行四个步骤：视觉分支（空间自注意力 + 时间自注意力 + 交叉文本注意力 + FFN/MoE）→ 音频分支（自注意力 + 交叉文本注意力 + FFN/MoE）→ CBGA 交叉模态桥（可选）→ LipSync 桥（可选）。

梯度检查点：在 DualBranchBlock 级别启用，节省约 90% 激活内存，额外计算开销约 20%。

3.3.3 VisionBranch — STDiT 视觉分支

子层	操作	描述
空间自注意力	逐帧 $H \times W$ 自注意力	reshape: $(B \times T, H \times W, D)$
时间自注意力	跨帧同位置注意力	reshape: $(B \times H \times W, T, D)$
交叉文本注意力	视频 token 查询文本	标准交叉注意力
FFN / MoE	逐 token 前馈	MLP 或 MoE

AdaLN 条件注入：每个子层有独立的 `AdaLN(shift, scale, gate)` 模块，时间步 t 通过 t_emb 调制所有归一化参数，`gate` 初始化为零以确保训练启动稳定。

3.3.4 AudioBranch — DiT 音频分支

子层	描述
自注意力	音频 token 自注意力（展平的频率×时间序列）
交叉文本注意力	音频 token 查询文本
FFN / MoE	逐 token 前馈

3.3.5 CBGA — 跨分支门控注意力桥

Seedance 2.0 的核心创新——视频和音频分支之间的双向通信：

```
Audio → v2a_attn(query: audio, key/value: video) → Audio (增强)
Video → a2v_attn(query: video, key/value: audio) → Video (增强)
```

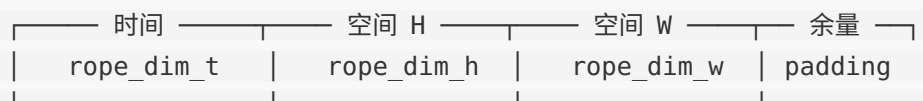
门控机制: 每个方向有可学习的标量 gate（零初始化）， $\text{gate 值} = \text{warmup_scale}(t) \times \text{learnable_gate} \times \text{sigmoid}(t_proj(t_emb))$ ，线性预热 50,000 步。

部署策略: CBGA 仅在特定层（如 24 层模型中的 [6, 12, 18]），其他层分支独立运行。

3.3.6 MM-RoPE — 多模态旋转位置嵌入

将头维度划分为独立的频率子空间，每个编码一个位置维度：

head_dim 分配：



不同的频率基使模型能够区分时间偏移、空间偏移和音频偏移。

3.3.7 MoE — 混合专家前馈网络

属性	值
路由策略	Top-K (K=2)
专家数量	32（可配置）
共享专家	是（DeepSeek 风格）
负载均衡	Switch Transformer 辅助损失
Router Z-loss	DeepSeek 风格 log-sum-exp 稳定化

3.3.8 MultiHeadAttention — 多头注意力

自动选择后端: xformers > flash-attn > PyTorch SDPA (通用回退)。QK 归一化: Q/K 上的 RMSNorm (源自 SD3/Flux 设计)。

3.3.9 模型变体总结

变体	层数	隐藏维度	头数	参数量	训练 VRAM
Small	12	768	12	~0.4B	~25GB
Base	24	1024	16	~1.6B	~72GB
30B Dense	48	2048	32	~30B	~160GB (8×A100)
30B MoE	48	2048	32	~30B	~80GB (4×A100)
200B MoE	48	4096	32	~200B	NVMe offload

4. 扩散框架 — Flow Matching

文件: [flux/diffusion/flow_matching.py](#)

4.1 理论

使用 **Rectified Flow** 框架 (Lipman et al., 2023; Esser et al., 2024)：

- 前向过程: $x_t = (1-t) \cdot x_0_{\text{noise}} + t \cdot x_1_{\text{clean}}$ ，其中 $t \in [0, 1]$
- 预测目标: 速度场 $v = x_1 - x_0 = \text{clean} - \text{noise}$
- 损失: $\text{MSE}(v_{\text{pred}}, v_{\text{target}})$

4.2 时间步采样

使用 **Logit-Normal 调度**: 采样 t 集中在中部区域 (噪声水平变化最大的区域)，比均匀采样更高效。

4.3 采样 (推理)

支持两种 ODE 求解器：

求解器	阶数	每次评估	特征
Euler	1 阶	1×	快速，质量可接受
Heun	2 阶	2× (预测 + 中点修正)	更高质量，默认选择

CFG（无分类器引导）：独立视频 CFG 尺度（默认 5.0），独立音频 CFG 尺度（默认 4.0）。

I2VA 首帧条件注入：首帧潜变量替换对应位置的噪声，首帧掩码强制模型预测零速度。

5. 训练系统

5.1 多阶段训练课程

阶段	描述	分辨率	步数
Stage 1	视频预训练（视觉分支）	256×256, 16-32帧	500K
Stage 2	音频预训练（音频分支）	16kHz mel	200K
Stage 3	AV 联合训练（激活 CBGA）	256×256, 16-32帧	300K
Stage 4	高分辨率微调	512×512, 64帧	100K

初始化策略：Stage 1 从 PixArt-α 初始化视觉分支，时间层零初始化，音频分支冻结；Stage 2 从零训练音频分支，视觉分支冻结；Stage 3 从各自阶段加载两个分支，CBGA gate 零初始化；Stage 4 从 Stage 3 继续，提升分辨率和帧数。

5.2 训练器设计

- **优化器**: AdamW，分组权重衰减（Bias 和 Norm 参数无衰减）
- **调度器**: Cosine + 线性预热（5000 步）
- **EMA**: 0.9999 衰减
- **混合精度**: bf16（AMP autocast），fp16 时启用 GradScaler
- **梯度累积**: `effective_batch = batch_size_per_gpu × num_gpus × grad_accum_steps`

5.3 分布式训练

特性	实现
启动器	torchrun（PyTorch 原生）
数据并行	DistributedSampler（每 epoch shuffle）
模型并行	FSDP (FULL_SHARD)，在 DualBranchBlock 上 auto-wrap
混合精度	FSDP MixedPrecision（param/reduce/buffer = bf16）
梯度检查点	FSDP 激活检查点，作用于 DualBranchBlock

自动检测模式: torchrun 多节点 → RANK/WORLD_SIZE/LOCAL_RANK 环境变量；单节点多 GPU → 自动检测 GPU 数量；单 GPU → 无分布式包装。

5.4 SFT — 监督微调

控制条件损失叠加在基础 Flow Matching 损失上：

损失	权重	描述
Flow Matching	1.0	基础视频+音频速度场预测
LFA 一致性	0.6	跨帧角色身份保留
KP 重建	0.4	面部 3D 关键点重建
AV 同步	0.1	音视频对齐对比损失
镜头控制	0.05	相机类型 + 景别嵌入一致性

5.5 RLHF — 强化学习微调

5 维奖励模型: visual_quality (0.20), motion_smoothness (0.25), character_consistency (0.25), av_sync (0.15), prompt_alignment (0.15)。

PPO 训练流程：从当前策略采样 K 个候选 → 奖励模型评分 → KL 惩罚 → PPO clip 更新 → 自适应调整 KL 系数。

6. 物理一致性设计

物理一致性是视频生成的核心挑战。扩散模型在视觉质量方面表现出色，但经常违反基本物理定律——物体消失、不自然的轨迹、缺少碰撞检测、错误的重力方向。Flux 通过**六种互补机制**系统性地解决这个问题，横跨四个阶段：预训练（数据标注）、训练（损失约束 + DPO）、后训练（探针监控）和推理（运动先验 + 轨迹注入）。

6.1 机制总览

机制	阶段	核心思想
VPT（角色感知）	预训练 + 训练	用物理角色标记对象 + 模态解耦噪声
World Model Loss	训练	未来帧预测 + 时间一致性 + 碰撞检测
PhysCorr（PhysicsRM + DPO）	训练	小型奖励模型评分 → DPO 偏好优化
PhysicsProbe	后训练监控	DiT 隐藏状态 → 线性解码物理合理性

机制	阶段	核心思想
PhaseLock	推理	2 步粗略去噪锁定运动先验，防止视觉精修覆盖
CausalMotion	推理	VLM 分解提示词 → 关键帧 + 轨迹，软约束注入

六种机制形成五层防御体系：数据层（VPT 角色标注）→ 损失层（World Model Loss + VPT 解耦噪声）→ 偏好层（PhysCorr DPO）→ 监控层（PhysicsProbe）→ 推理层（PhaseLock + CausalMotion）。

7. 数据管道

7.1 数据工程理念

数据管道旨在生成一个**平衡、caption 丰富**的训练集，包含多样化的人类场景。早期实验表明，VoxCeleb2 主导的 manifest（99.75% 为对话人脸）导致模型退化为"模糊的对话人脸"并停滞在 $\text{video_loss} \approx 0.011$ 。当前管道通过以下方式解决此问题：

- 1. **多源聚合** — 8 个不同数据源，控制比例
- 2. **VLM captioning** — Qwen2-VL-2B 用真实描述替换模板 caption
- 3. **运动增强** — 静态图像（CelebA-HQ）获得合成相机运动
- 4. **质量过滤** — 每个视频经过光流 + 分辨率 + 时长三重门控
- 5. **声明式 manifest 构建器** — 单个脚本组装和平衡所有来源

7.2 训练数据组成

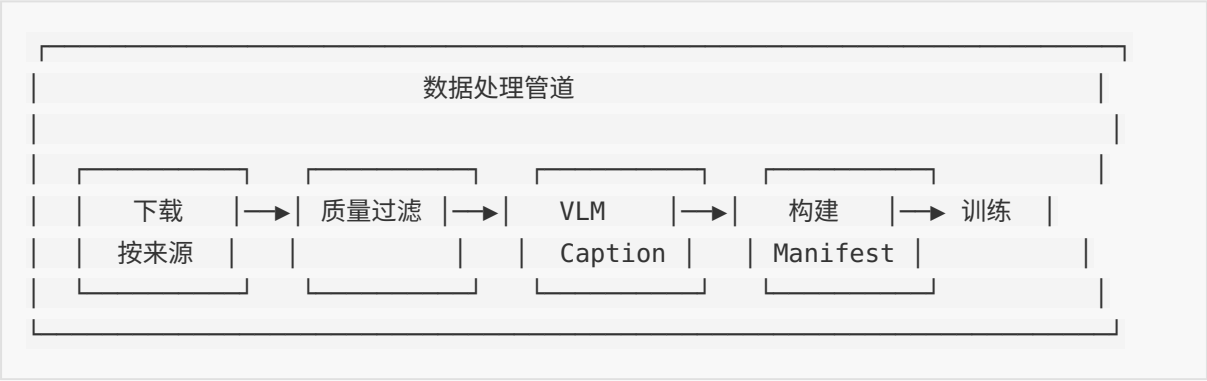
当前 Stage 1 训练集：**76k 视频，约 850 小时，22k 种独特 caption。**

数据来源	视频数	占比	分辨率	Caption 来源	下载工具
CelebA-HQ	28,560	37.5%	256×256	原始（8.4k 种）	<code>scripts/ extract_celeba_hq.py</code>
VoxCeleb2	14,220	18.7%	224×224	10 种多样化模板	<code>download_voxceleb.py</code>
YouTube	13,098	17.2%	640×360+	Qwen2-VL-2B	<code>scrapers/ youtube_scraper.py</code>
Bilibili	11,540	15.5%	720×1280	Qwen2-VL-2B	<code>scrapers/ bilibili_download.py</code>
WebVid	7,593	10.0%	596×336	Shutterstock captions	<code>download_webvid.py</code>

数据来源	视频数	占比	分辨率	Caption 来源	下载工具
Animation	1,581	2.1%	混合	模板	<code>scrapers/ bilibili_download.py</code>
Pexels	~800	1.0%	1080p-4K	Qwen2-VL-2B	<code>download_pexels.py</code>
HDTF	330	0.4%	360×640	原始	<code>download_hdtf.py</code>

Caption 质量至关重要。模板 caption ("一个人在对镜头说话") 无法教会模型文本到内容的映射。真实的 VLM caption ("一位长棕色头发的年轻女性在柔和的自然窗光中对着镜头微笑，中景肖像") 才能教会。约 30% 的训练数据有 Qwen2-VL-2B 生成的 VLM caption。

7.3 数据处理管道



第一步：下载

每个来源有专用工具：

来源	方法	备注
YouTube	yt-dlp 搜索（无需 API key）	~300 组关键词查询，覆盖日常生活、运动、职业、情绪、地点
Bilibili	Bilibili 搜索 API	25 组标签（每组 5 个关键词），覆盖人像、vlog、时尚、美食、宠物等
CelebA-HQ	HuggingFace 数据集	<code>Ryan-sjtu/celebahq-caption</code> ：30k 张图像 + parquet 格式 caption
WebVid	WebVid-10M 元数据	49,500 个 Shutterstock caption 已缓存，~8k 视频已下载
Pexels	Pexels/Pixabay API	免费素材，~800 个高分辨率视频，覆盖 6 个类别

来源	方法	备注
VoxCeleb2	官方来源	1.09M 个片段已下载，经质量评估后使用 15k (1.4%)

第二步：质量过滤

flux/tools/quality_filter.py 应用三重门控：

门控	阈值	拒绝
分辨率	$h \geq 360\text{px}, w \geq 360\text{px}$	缩略图、低分辨率代理
时长	$2\text{s} \leq \text{dur} \leq 300\text{s}$	闪屏、电影、Shorts
光流	$0.05 \leq \text{flow} \leq 8.0$	静态幻灯片、混乱故障

YouTube 通过率约 75%，Bilibili 约 65%。

第三步：VLM Captioning

scripts/caption_videos_vlm.py 使用 Qwen2-VL-2B（本地缓存）：

- 1. 每个视频均匀采样 4 帧，调整大小至最大 448px
- 2. 以结构化提示词馈送给 VLM，要求描述人物/场景
- 3. 每个视频约 0.5s 推理，速度 2.0 vids/sec
- 4. 输出：data/manifests/vlm_captions.json（约 11k caption）
- 5. 自动续传（--resume 标志）— 跳过已 caption 的视频

Caption 质量前后对比：

之前（模板）	之后（VLM）
"一个年轻女子在明亮的房间里行走"	"一位长棕色头发的年轻女性在柔和的自然窗光中对着镜头微笑，中景肖像"
"一个人在现代环境中坐着"	"一位留着短发和胡须的年轻男子，穿着黑色背心和蓝色牛仔裤，坐在咖啡馆里，背景有窗户"

模型选择： Qwen2-VL-2B-Instruct（约 4 GB VRAM，约 2 vids/sec）。如需更高质量，可切换至 Qwen2-VL-7B-Instruct（约 14 GB）。模型以 local_files_only=True 加载，首次下载后可离线推理。

第四步：构建 Manifest

`scripts/build_balanced_manifest.py` 组装最终训练 CSV，按优先级选择 caption: VLM captions → 来源 captions → 多样化模板。

7.4 CelebA-HQ 运动增强

3 万张静态人脸图像通过 `scripts/extract_celeba_hq.py` 转换为 32 帧伪视频。如果不这样做，模型会学到"视频没有帧间变化"并产生静态噪声。

每个伪视频的增强参数（随机化）：

参数	范围	模拟效果
缩放	0.90-1.12× 正弦波	呼吸 / 前倾
平移	6-12% 平滑随机路径	相机摇摆
旋转	±1-3° 漂移	手持不稳
亮度	±2-6% 波动	曝光变化

每个伪视频获得独特的随机运动参数。帧间像素差异平均 18-25（与真实手持拍摄匹配）。

7.5 数据加载器

- **VideoDataset:** CSV Manifest → OpenCV 视频解码 → 随机起始帧裁剪 → 数据增强。支持 resize、随机裁剪、水平翻转、颜色抖动。CFG caption dropout 10%。错误恢复：失败的样本回退到随机其他样本。
- **AudioDataset:** 读取音频 → 重采样至 16kHz → 梅尔频谱 → log 压缩
- **AVDataset:** 联合视频+音频加载，支持 I2VA 首帧条件（30% 概率），CFG dropout 10%

8. 推理管道

8.1 T2VA — 文本到视频+音频

Prompt → T5 编码器 → Flow Matching ODE (30 步 Heun)
→ VideoVAE 解码器 → 视频帧 (T, C, H, W)
→ AudioVAE 解码器 → 音频波形

8.2 I2VA — 图像到视频+音频

图像 + 提示词 → VideoVAE 编码 (首帧) + T5 编码
→ Flow Matching ODE (首帧条件注入)
→ VideoVAE 解码 + AudioVAE 解码

8.3 级联 — 4K 30 秒

- Stage A (粗粒度) : 32帧 × 256×256, 30 步 — 粗略结构生成
- Stage B (时间) : 32帧 → 120帧, 10 步 — 时间插值 + 扩散精修
- Stage C (空间) : 256 → 512 → 1024 → 4K, 每阶段 10 步 — 空间超分辨率

9. 损失函数

损失	用途
Flow Matching Loss	速度场 MSE + 可选 AV 同步对比损失
VAE Loss	重建 (L1+LPIPS) + KL + GAN (3D PatchGAN)
Lip Sync Loss	口型同步交叉注意力
Sync Loss	AV 对齐对比损失 (InfoNCE 风格)
World Model Loss	未来帧预测 + 时间一致性

Flow Matching 损失公式:

```
video_loss = MSE(v_pred, x1_v - x0_v)
audio_loss = MSE(a_pred, x1_a - x0_a)
total      = w_v × video_loss + w_a × audio_loss
           + load_balance_loss + router_z_loss (MoE)
```

10. 配置系统

OmegaConf YAML 配置; 每个训练阶段有独立的配置文件。

关键配置项:

```
training:
  model:
```

```
dim: 1024
num_layers: 24
num_heads: 16
data:
  manifest_path: "data/manifests/train_full_train.csv"
  resolution: 256
  num_frames: 32
optimizer: {type: adamw, lr: 2.0e-4, weight_decay: 0.01}
scheduler: {type: cosine, warmup_steps: 5000}
max_steps: 500000
batch_size: 8
gradient_accumulation_steps: 2
```

11. 附加设计特性

11.1 LipSync — 唇形同步桥

CBGA 之外的额外跨模态注意力，聚焦于嘴部区域：MouthRegionMask（高斯注意力掩码）、MouthROIAttention（视频嘴部 token 通过交叉注意力查询音频 token）、Viseme Embedding（14 类 MPEG-4 视素嵌入）。

11.2 级联管道 — 4K 30 秒生成

通过三阶段级联实现超长超高清视频生成，避免一次性生成的计算爆炸。

12. 项目目录结构

```
flux/
├─ configs/                # YAML 配置
│   └─ inference/          # T2VA/I2VA 推理配置
│   └─ model/              # 模型架构配置
│   └─ train/              # 训练阶段配置
├─ scripts/               # 入口脚本
│   └─ train.sh            # 训练启动器
│   └─ infer.py            # 快速推理测试
│   └─ build_balanced_manifest.py # 均衡 manifest 构建器
│   └─ extract_celeba_hq.py # CelebA-HQ 图像→伪视频
│   └─ caption_videos_vlm.py # Qwen2-VL 视频 captioning
│   └─ download_celeba_hq.py # CelebA-HQ 下载器
├─ scrapers/              # 网络数据采集
│   └─ youtube_scraper.py  # YouTube 下载 (yt-dlp)
```

```
| | |─ bilibili_download.py # Bilibili 下载
| | └─ douyin_scraper.py   # 抖音采集
| └─ flux/                  # 主包
| | └─ models/              # 模型定义
| | | └─ video_vae/         # VideoVAE
| | | └─ audio_vae/         # AudioVAE
| | | └─ db_dit/            # DB-DiT
| | | └─ text_encoder/      # T5 编码器
| | | └─ ...
| | └─ diffusion/           # 扩散框架
| | └─ data/                # 数据管道
| | └─ training/            # 训练系统
| | └─ loss/                # 损失函数
| | └─ pipelines/           # 推理管道
| | └─ physics/             # 物理一致性
| └─ tools/                 # 数据准备 CLI 工具
└─ docs/                   # 文档
  └─ README.md
```

13. 测试策略

测试文件	覆盖范围
test_db_dit.py	DB-DiT 前向/反向传播，输出形状
test_flow_matching.py	Flow Matching 损失 + 采样 + CFG
test_video_vae.py	VideoVAE 编码/解码，KL 损失，SDXL 初始化
test_mm_rope.py	MM-RoPE 3D + 1D 旋转
test_cbga.py	CBGA 门控，预热，双向注意力
test_physics.py	Physics Probe 训练和推理
test_pipeline.py	端到端 T2VA / I2VA 管道

14. 性能基准

14.1 训练吞吐量

配置	GPU	有效批次	Steps/s	500K 步训练时间
Small	1× A100 40GB	4	~1.2	~5 天
Base	4× A100 80GB	16	~0.8	~8 天
Base	8× A100 80GB	128	~1.0	~6 天
30B MoE	8× A100 80GB	128	~0.3	~20 天

14.2 推理速度

配置	帧数	分辨率	采样步数	推理时间 (A100)
Base, Euler	32	256×256	20	~15s
Base, Heun	32	256×256	30	~40s
Cascade 4K	128	3840×2160	50	~10min

15. 参考文献

- [Seedance 2.0: Advancing Video Generation for World Complexity](#) — ByteDance Seed Team, 2026
- [Seedance 1.5 Pro: A Native Audio-Visual Joint Generation Foundation Model](#) — ByteDance Seed Team, 2025
- [Flow Matching for Generative Modeling](#) — Lipman et al., 2023
- [Scaling Rectified Flow Transformers for High-Resolution Image Synthesis](#) — Esser et al., 2024
- [Open-Sora: Democratizing Efficient Video Production](#) — HPC-AI Tech
- [DeepSeek-V2/MoE: A Strong, Economical Mixture-of-Experts Language Model](#) — DeepSeek, 2024
- [The Invisible Hand of Physics in Video Diffusion Models](#) — Esmati et al., 2026